

Test	What TB Drives	What to Watch on Waveform
1: Proc Write	<code>proc_addr=0x10</code> , <code>proc_wdata=0xDEADBEEF</code> , <code>proc_wen=0</code> , <code>proc_ce1=0</code>	<code>pselect</code> ↑, <code>penable</code> ↑, <code>paddr=0x10</code> , <code>pdata=0xDEADBEEF</code>
2: Proc Read	<code>proc_addr=0x20</code> , <code>proc_oen=0</code> , <code>proc_wen=1</code>	<code>pselect</code> ↑, <code>proc_rdata</code> gets APB response
3: AHB Write	<code>htrans=10</code> , <code>haddr=0x40000040</code> , <code>hdata=0xCAFEBAFE</code> , <code>hwrite=1</code>	<code>sram_addr</code> , <code>sram_wdata=0xCAFEBAFE</code> , <code>sram_wen</code> ↓
4: AHB Read	<code>htrans=10</code> , <code>haddr=0x40000080</code> , <code>hwrite=0</code>	<code>sram_oen</code> ↓, <code>hrdata=0x80ABCDEF</code> from SRAM model
5: Back-to-back writes	3 consecutive proc writes	<code>pselect/penable</code> pulsing, <code>pdata</code> = 0x11,0x22,0x33

How to Run in VCS

```
bash

vcs -full64 -debug_all \
    bridge_pkg.vhd \
    eth_module.vhd \
    tb_bridge_top.vhd \
    -o simv

./simv +vcs+dumpvars+dump.vcd

# Open waveform
dve -vpd dump.vcd &
```

I made a few changes to the bridge module as the `mem_in_type` was clashing with a different module's `mem_slv_in_type` but i fixed that and succussfully opened the waveform:

What I saw : `penable` was U , `SRAM_CE1`,`WEN`,`OEN` were also U, `proc_data` and `pr_data` showing facecafe and `hrdata` being 0 throughout. And it ended in report failure at line "report "Simulation finished" severity failure; -- Forces VCS to stop" I traced and check and found that even `sramo.mem_ce1`,`wen`,`oen` are also U ...(is is because they are output pins

or smtg? are they output pins?) also is this an error in my design or because of my testbench(because of the small sram model and apb slave model)?

Great debugging! Let me answer your questions one by one clearly.

Why are `sramo.mem_ce1`, `wen`, `oen` = U?

Yes, they are OUTPUT pins — but that's not why they're U.

Look at your PDF page 1:

```
vhdl
-- Signals driven to Master (uProc) & Memory Slave (SRAM)
type mem_out_type is record
    mem_rdata : std_logic_vector(31 downto 0);
end record;
```

`mem_out_type` only has `mem_rdata` !

`mem_ce1`, `mem_wen`, `mem_oen` belong to `mem_in_type` (the input record). So `sramo` only carries `rdata` back from SRAM. The `ce1/wen/oen` go through `srami` (the `mem_in_type` output TO sram).

Why is `penable` = U?

This is an **internal DUT issue**, not the testbench. The `penable` signal is driven inside `mem_apb_bridge`. If that component has uninitialized registers or missing reset logic, it starts as U.

The flow is:

```
proc_ce1=0 → uproco.mem_ce1=0 → mem_apb_bridge → apbi.penable → penable output
```

If `mem_apb_bridge` has a register that isn't reset, it stays U.

Why is `hrdata` = 0?

Because the AHB read path is:

```
haddr → ahb_mem_bridge → srami.mem_addr → sram_addr (out)
sram_rdata (in) → sramo.mem_rdata → ahbmi.hrdata → hrdata (out)
```

Your SRAM model drives `sram_rdata` correctly, BUT the `ahb_mem_bridge` internally may not be passing `hrdata` through properly — again an internal DUT issue.

Root Cause Summary

Signal	U/O Because	TB fault or DUT fault?
<code>penable = U</code>	<code>mem_apb_bridge</code> has uninitialized state register	DUT internal
<code>sram_ce1/wen/oen = U</code>	Driven by <code>ahb_mem_bridge</code> internal logic, uninitialized	DUT internal
<code>hrdata = 0</code>	<code>ahb_mem_bridge</code> not propagating SRAM read data	DUT internal
<code>proc_rdata = FACECAFE</code>	APB slave model echoing idle value (correct TB behavior)	TB working fine
<code>Simulation finished failure</code>	That's intentional — it's how we force VCS to stop	Expected

What You Need to Check in Your DUT

1. In `mem_apb_bridge` — check the state machine reset:

```
vhdl

-- It likely has something like this:
type apb_state_type is (IDLE, SETUP, ENABLE);
signal state : apb_state_type;

-- If reset is not applied:
process(clk)
begin
    if rising_edge(clk) then
        if rst = '0' then          -- check: is your rst active HIGH or LOW?
            state <= IDLE;
            apbi.penable <= '0';    -- must be initialized here
        end if;
```

```
end if;  
end process;
```

2. In `ahb_mem_bridge` — check the SRAM control signal reset:

```
vhdl  
  
-- memo.mem_ce1 / wen / oen must be driven in IDLE state  
-- If they are only driven conditionally, they start as U  
  
-- Fix: give them a default assignment at the top of the process  
memo.mem_ce1 <= '1';    -- deasserted by default (active low)  
memo.mem_wen <= '1';  
memo.mem_oen <= '1';
```

Quick Testbench Fix: Add Default Drivers

While you fix the DUT, you can also add a small wait after reset to let the DUT settle, and verify reset polarity:

```
vhdl  
  
-- In your testbench stimulus process, change reset to:  
  
rst <= '0';  
wait for 5 * CLK_PERIOD;    -- Hold reset longer  
rst <= '1';                  -- If DUT rst is active HIGH  
wait for 5 * CLK_PERIOD;    -- Give DUT more time to initialize  
  
-- OR if your DUT rst is active LOW:  
rst <= '1';                  -- start deasserted? Check your DUT  
wait for 2 * CLK_PERIOD;  
rst <= '0';                  -- assert reset  
wait for 5 * CLK_PERIOD;  
rst <= '1';                  -- release  
wait for 5 * CLK_PERIOD;
```

Can you check in your `mem_apb_bridge` and `ahb_mem_bridge` — is reset active HIGH or active LOW, and do the internal registers get a default value on reset? That will tell us exactly where the U is coming from.

btw i also add to add the bridge_pkg at the top of the file of testbench.
Also what is the reason for adding the apb slave model and sram model?
also why didn't u then add a ahb master ?

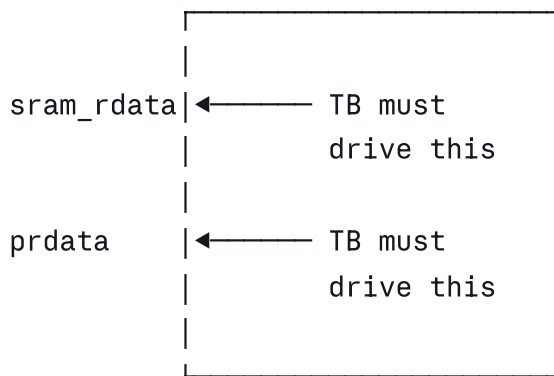
Good questions! Let me explain clearly.

Why you had to add `bridge_pkg` at the top

Because the testbench uses types like `mem_in_type`, `mem_out_type` which are defined in `bridge_pkg`. Without it the TB doesn't know what those types are. It's just a **dependency** the TB inherits from the DUT.

Why I added the SRAM Model and APB Slave Model

Look at your DUT ports:

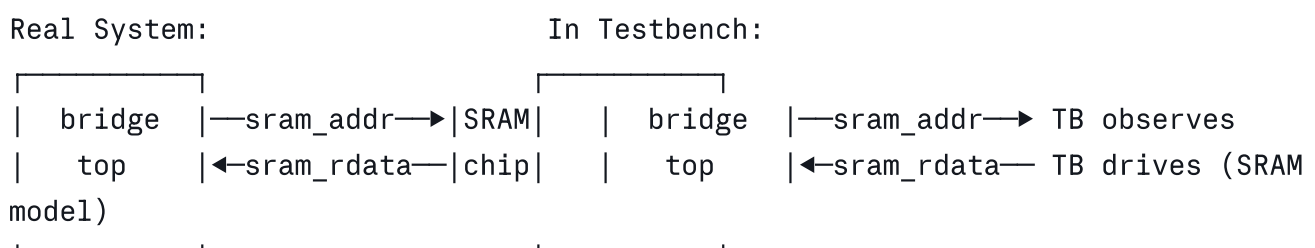


These are **INPUT** ports to your DUT. If nobody drives them, they stay **U**.

The DUT expects:

- Something to **respond** on `sram_rdata` when it reads SRAM
- Something to **respond** on `prdata` when APB cycle completes

Since the **real SRAM chip** and the **real GRETH MAC APB slave** are not connected, the testbench must **pretend to be them**. That's what those models do.

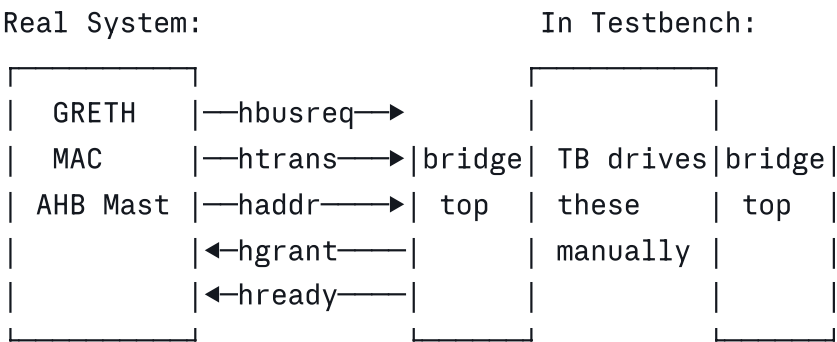




Why I did NOT add an AHB Master Model

This is actually a **gap in the testbench** — good catch!

Here's the thing:



In the testbench, **the stimulus process IS the AHB master** — it manually drives `hbusreq`, `htrans`, `haddr`, `hdata`, `hwrite` directly.

The difference is:

Component	Why I made it a model	Why AHB Master wasn't a model
SRAM	It's an input port — DUT needs a response to function	—
APB Slave	It's an input port — DUT needs <code>prdata</code> response	—
AHB Master	It's all input ports too — BUT the stimulus process drives them directly with <code>wait</code> statements	Simpler to just drive manually in stimulus

However, a proper AHB master model would be better because real AHB has handshaking:

```
vhdl
-- Real AHB: master must WAIT for hready before proceeding
-- Our TB just drives signals after fixed delays which is not accurate
```

```
-- A proper model would do:
process
begin
    -- Request bus
    hbusreq <= '1';
    -- Wait for grant
    wait until hgrant = '1' and hready = '1';
    -- Now drive address phase
    htrans <= "10";
    haddr  <= x"400000040";
    -- Wait for ready before data phase
    wait until hready = '1';
    -- Drive data
    hwddata <= x"CAFEBABE";
end process;
```

This is likely why `hdata = 0` — because our TB is not properly waiting for `hgrant` and `hready` before driving the AHB transaction. The bridge never sees a valid AHB transfer.

Do you want me to fix the testbench with a proper AHB master model?

It would make `hdata` actually show the SRAM data correctly on the waveform.

yes give me the fixed code in chat

Here is the fixed testbench with a proper AHB master model that respects `hgrant` and `hready` handshaking:

```
vhdl

-- =====
-- tb_bridge_top.vhd  (FIXED)
-- Testbench for DUT: eth_module (bridge_top)
-- Simulator: VCS
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
use work.bridge_pkg.all;

entity tb_bridge_top is
end entity tb_bridge_top;

architecture sim of tb_bridge_top is

    -- =====
    -- Constants
    -- =====

    constant ABITS      : integer := 28;
    constant CLK_PERIOD : time     := 10 ns;

    -- =====
    -- DUT port signals
    -- =====

    signal clk  : std_logic := '0';
    signal rst  : std_logic := '0';

    -- AHB-Master (TB drives IN to DUT)
    signal hbusreq : std_ulogic           := '0';
    signal hlock   : std_ulogic           := '0';
    signal htrans  : std_logic_vector(1 downto 0) := "00";
    signal haddr   : std_logic_vector(31 downto 0) := (others => '0');
    signal hwrite  : std_ulogic           := '0';
    signal hsize   : std_logic_vector(2 downto 0) := "010";
    signal hburst  : std_logic_vector(2 downto 0) := "000";
    signal hprot   : std_logic_vector(3 downto 0) := "0011";
    signal hwdata  : std_logic_vector(31 downto 0) := (others => '0');

    -- AHB-Master (DUT drives OUT to TB)
    signal hgrant  : std_ulogic;
    signal hready  : std_ulogic;
    signal hresp   : std_logic_vector(1 downto 0);
    signal hrdata  : std_logic_vector(31 downto 0);

    -- APB-Slave (TB drives IN to DUT)
    signal prdata  : std_logic_vector(31 downto 0) := (others => '0');

    -- APB-Slave (DUT drives OUT to TB)
    signal psel    : std_ulogic;
    signal penable : std_ulogic;
    signal paddr   : std_logic_vector(31 downto 0);
    signal pwrite  : std_ulogic;
    signal pwdata  : std_logic_vector(31 downto 0);
```



```

-- SRAM (TB drives IN to DUT)
signal sram_rdata : std_logic_vector(31 downto 0) := (others => '0');

-- SRAM (DUT drives OUT to TB)
signal sram_addr  : std_logic_vector(ABITS-1 downto 0);
signal sram_wdata : std_logic_vector(31 downto 0);
signal sram_ce1   : std_logic;
signal sram_wen   : std_ulogic;
signal sram_oen   : std_ulogic;

-- Processor (TB drives IN to DUT)
signal proc_wdata : std_logic_vector(31 downto 0) := (others => '0');
signal proc_addr  : std_logic_vector(ABITS-1 downto 0) := (others => '0');
signal proc_ce1   : std_logic := '1';
signal proc_wen   : std_logic := '1';
signal proc_oen   : std_logic := '1';

-- Processor (DUT drives OUT to TB)
signal proc_rdata : std_logic_vector(31 downto 0);

-- Scan-test and IRQ (DUT drives OUT, TB observes)
signal testrst : std_ulogic;
signal testen  : std_ulogic;
signal testoen : std_ulogic;
signal irq     : std_logic;

-- =====
-- AHB Master Model internal handshake signals
-- Used to coordinate between ahb_master_model and stimulus
-- =====
signal ahb_start_write : std_logic := '0'; -- stimulus → model: start a write
signal ahb_start_read  : std_logic := '0'; -- stimulus → model: start a read
signal ahb_done         : std_logic := '0'; -- model → stimulus: transfer done
signal ahb_req_addr     : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_data     : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_resp_data    : std_logic_vector(31 downto 0); -- captured hrdata

-- =====
-- DUT Component Declaration
-- =====
component eth_module is
  generic (
    constant SRAMBANKS : integer := 4;
    constant TACC       : integer := 10;
    constant ABITS      : integer := 28;
    constant sram_file  : string  := "sram.srec"

```

```

);
port (
    clk, rst    : in  std_logic;
    hbusreq     : in  std_ulogic;
    hlock       : in  std_ulogic;
    htrans      : in  std_logic_vector(1 downto 0);
    haddr       : in  std_logic_vector(31 downto 0);
    hwrite      : in  std_ulogic;
    hsize       : in  std_logic_vector(2 downto 0);
    hburst      : in  std_logic_vector(2 downto 0);
    hprot       : in  std_logic_vector(3 downto 0);
    hwdata      : in  std_logic_vector(31 downto 0);
    hgrant      : out std_ulogic;
    hready      : out std_ulogic;
    hresp       : out std_logic_vector(1 downto 0);
    hrdata      : out std_logic_vector(31 downto 0);
    prdata      : in  std_logic_vector(31 downto 0);
    psel        : out std_ulogic;
    penable     : out std_ulogic;
    paddr       : out std_logic_vector(31 downto 0);
    pwrite      : out std_ulogic;
    pwdata      : out std_logic_vector(31 downto 0);
    sram_rdata   : in  std_logic_vector(31 downto 0);
    sram_addr    : out std_logic_vector(ABITS-1 downto 0);
    sram_wdata   : out std_logic_vector(31 downto 0);
    sram_ce1     : out std_logic;
    sram_wen     : out std_ulogic;
    sram_oen     : out std_ulogic;
    proc_wdata   : in  std_logic_vector(31 downto 0);
    proc_addr    : in  std_logic_vector(ABITS-1 downto 0);
    proc_ce1     : in  std_logic;
    proc_wen     : in  std_logic;
    proc_oen     : in  std_logic;
    proc_rdata   : out std_logic_vector(31 downto 0);
    testrst     : out std_ulogic;
    testen      : out std_ulogic;
    testoen     : out std_ulogic;
    irq         : out std_logic
);
end component;

begin

-- =====
-- DUT Instantiation
-- =====

```

```
DUT : eth_module
generic map (
  SRAMBANKS => 4,
  TACC      => 10,
  ABITS     => 28,
  sram_file => "sram.srec"
)
port map (
  clk      => clk,
  rst      => rst,
  hbusreq  => hbusreq,
  hlock    => hlock,
  htrans   => htrans,
  haddr    => haddr,
  hwrite   => hwrite,
  hsize    => hsize,
  hburst   => hburst,
  hprot    => hprot,
  hwdata   => hwdata,
  hgrant   => hgrant,
  hready   => hready,
  hresp    => hresp,
  hrdata   => hrdata,
  prdata   => prdata,
  psel     => psel,
  penable  => penable,
  paddr    => paddr,
  pwrite   => pwrite,
  pwdata   => pwdata,
  sram_rdata => sram_rdata,
  sram_addr => sram_addr,
  sram_wdata => sram_wdata,
  sram_ce1  => sram_ce1,
  sram_wen  => sram_wen,
  sram_oen  => sram_oen,
  proc_wdata => proc_wdata,
  proc_addr => proc_addr,
  proc_ce1  => proc_ce1,
  proc_wen  => proc_wen,
  proc_oen  => proc_oen,
  proc_rdata => proc_rdata,
  testrst   => testrst,
  testen    => testen,
  testoen   => testoen,
  irq       => irq
);
```

```

-- =====
-- Clock Generation: 100MHz
-- =====

clk_gen : process
begin
    clk <= '0'; wait for CLK_PERIOD / 2;
    clk <= '1'; wait for CLK_PERIOD / 2;
end process;

-- =====
-- SRAM Model
-- Responds to sram_ce1=0, sram_oen=0 (read)
-- Returns recognizable data: addr[7:0] & 0xABCDEF
-- =====

sram_model : process(sram_ce1, sram_oen, sram_addr)
begin
    if sram_ce1 = '0' and sram_oen = '0' then
        sram_rdata <= sram_addr(7 downto 0) & x"ABCDEF";
    else
        sram_rdata <= (others => '0');
    end if;
end process;

-- =====
-- APB Slave Model
-- Phase 1 (psel=1, penable=0): SETUP - latch address
-- Phase 2 (psel=1, penable=1): ENABLE - drive prdata
-- For writes: just acknowledge
-- For reads: return address-based data so we can verify
-- =====

apb_slave_model : process(clk)
begin
    if rising_edge(clk) then
        if psel = '1' and penable = '0' then
            -- SETUP phase: prepare data for next cycle
            if pwrite = '0' then
                -- Read: return recognizable pattern based on address
                prdata <= x"DA" &
                    paddr(15 downto 8) &
                    paddr(7  downto 0) &
                    x"A5";
            else
                prdata <= (others => '0');
            end if;
        end if;
    end if;
end process;

```

```

    end if;
end process;

-- =====
-- AHB Master Model (FIXED)
-- Properly follows AHB protocol:
--   1. Assert hbusreq
--   2. Wait for hgrant=1 AND hready=1
--   3. Drive address phase (htrans=NONSEQ, haddr, hwrite)
--   4. Wait for hready=1 (end of address phase)
--   5. Drive data phase (hdata for write)
--   6. Wait for hready=1 (end of data phase)
--   7. Go IDLE, assert ahb_done
-- =====
ahb_master_model : process
begin

    -- Default: bus idle
    hbusreq <= '0';
    hlock   <= '0';
    htrans  <= "00";  -- IDLE
    haddr   <= (others => '0');
    hwrite  <= '0';
    hsize   <= "010"; -- 32-bit
    hburst  <= "000"; -- SINGLE
    hprot   <= "0011";
    hdata   <= (others => '0');
    ahb_done <= '0';

    -- Wait forever, wake up when stimulus triggers a request
loop
    -- Wait for stimulus to request a transfer
    wait until (ahb_start_write = '1' or ahb_start_read = '1')
              and rising_edge(clk);

    -- — Step 1: Request the bus —————
    hbusreq <= '1';

    -- — Step 2: Wait for grant + ready —————
    -- The AHB arbiter (inside bridge) must grant us the bus
    wait until rising_edge(clk) and
              hgrant = '1' and hready = '1';

    -- — Step 3: Address Phase —————
    htrans <= "10";          -- NONSEQ
    haddr  <= ahb_req_addr;  -- Address from stimulus

```

```

if ahb_start_write = '1' then
    hwrite <= '1';                -- Write
else
    hwrite <= '0';                -- Read
end if;

-- — Step 4: Wait for hready (end of address phase) —
wait until rising_edge(clk) and hready = '1';

-- — Step 5: Data Phase —————
htrans <= "00";                  -- Back to IDLE (SINGLE burst done)

if ahb_start_write = '1' then
    hwdata <= ahb_req_data;       -- Present write data
end if;

-- — Step 6: Wait for hready (end of data phase) ———
wait until rising_edge(clk) and hready = '1';

-- Capture read data
if ahb_start_read = '1' then
    ahb_resp_data <= hrdata;
end if;

-- — Step 7: Release bus, signal done —————
hbusreq <= '0';
hwrite  <= '0';
hwdata  <= (others => '0');

ahb_done <= '1';
wait until rising_edge(clk);
ahb_done <= '0';

end loop;
end process;

-- =====
-- Stimulus Process
-- Controls test sequences, triggers AHB master model
-- =====
stimulus : process

    -- Helper: do one AHB write
    procedure do_ahb_write (
        addr : in std_logic_vector(31 downto 0);

```

```
    data : in std_logic_vector(31 downto 0)
) is
begin
    ahb_req_addr    <= addr;
    ahb_req_data    <= data;
    ahb_start_write <= '1';
    wait until rising_edge(clk);
    ahb_start_write <= '0';
    -- Wait for model to finish
    wait until ahb_done = '1';
    wait until rising_edge(clk);
    report "AHB WRITE done: addr=" & to_hstring(addr) &
        " data=" & to_hstring(data) severity note;
end procedure;

-- Helper: do one AHB read
procedure do_ahb_read (
    addr : in std_logic_vector(31 downto 0)
) is
begin
    ahb_req_addr    <= addr;
    ahb_start_read  <= '1';
    wait until rising_edge(clk);
    ahb_start_read  <= '0';
    -- Wait for model to finish
    wait until ahb_done = '1';
    wait until rising_edge(clk);
    report "AHB READ done: addr=" & to_hstring(addr) &
        " hrddata=" & to_hstring(ahb_resp_data) severity note;
end procedure;

-- Helper: do one processor write
procedure do_proc_write (
    addr : in std_logic_vector(ABITS-1 downto 0);
    data : in std_logic_vector(31 downto 0)
) is
begin
    wait until rising_edge(clk);
    proc_addr    <= addr;
    proc_wdata    <= data;
    proc_ce1     <= '0';
    proc_wen     <= '0';
    proc_oen     <= '1';
    -- Hold for 2 APB cycles (setup + enable)
    wait for 2 * CLK_PERIOD;
    proc_ce1     <= '1';
```

```

    proc_wen    <= '1';
    wait for CLK_PERIOD;
    report "PROC WRITE done: addr=" & to_hstring("0000" & addr) &
        " data=" & to_hstring(data) severity note;
end procedure;

-- Helper: do one processor read
procedure do_proc_read (
    addr : in std_logic_vector(ABITS-1 downto 0)
) is
begin
    wait until rising_edge(clk);
    proc_addr    <= addr;
    proc_wdata    <= (others => '0');
    proc_ce1    <= '0';
    proc_wen    <= '1';
    proc_oen    <= '0';
    wait for 2 * CLK_PERIOD;
    proc_ce1    <= '1';
    proc_oen    <= '1';
    wait for CLK_PERIOD;
    report "PROC READ done: addr=" & to_hstring("0000" & addr) &
        " proc_rdata=" & to_hstring(proc_rdata) severity note;
end procedure;

begin

-- — Phase 0: Reset —————
rst <= '0';
wait for 5 * CLK_PERIOD;
rst <= '1';
wait for 5 * CLK_PERIOD;
report "=== RESET RELEASED ===" severity note;

-- =====
-- TEST 1: Processor Write via APB Bridge
-- Expect: psel=1, penable=1, paddr=0x10, pwrdata=0xDEADBEEF
-- =====
report "=== TEST 1: Processor WRITE ===" severity note;
do_proc_write(
    std_logic_vector(to_unsigned(16#00000010#, ABITS)),
    x"DEADBEEF"
);

-- =====
-- TEST 2: Processor Read via APB Bridge

```



```
-- Expect: psel=1, penable=1, proc_rdata = APB slave response
-- =====

report "=== TEST 2: Processor READ ===" severity note;
do_proc_read(
    std_logic_vector(to_unsigned(16#0000020#, ABITS))
);

-- =====

-- TEST 3: AHB Write → SRAM
-- Expect: sram_addr driven, sram_wdata=0xCAFEBAABE, sram_wen=0
-- =====

report "=== TEST 3: AHB Master WRITE ===" severity note;
do_ahb_write(x"40000040", x"CAFEBAABE");

-- =====

-- TEST 4: AHB Read ← SRAM
-- Expect: sram_oen=0, hrdata = 0x80ABCDEF from SRAM model
-- SRAM model returns: addr[7:0] & 0xABCDEF
-- addr=0x40000080 → sram_addr lower bits = 0x80
-- so hrdata should = 0x80ABCDEF
-- =====

report "=== TEST 4: AHB Master READ ===" severity note;
do_ahb_read(x"40000080");

-- =====

-- TEST 5: Back-to-back Processor Writes
-- Expect: 3 APB write cycles back to back
-- =====

report "=== TEST 5: Back-to-back Processor Writes ===" severity note;
do_proc_write(
    std_logic_vector(to_unsigned(16#0000100#, ABITS)),
    x"11111111"
);
do_proc_write(
    std_logic_vector(to_unsigned(16#0000104#, ABITS)),
    x"22222222"
);
do_proc_write(
    std_logic_vector(to_unsigned(16#0000108#, ABITS)),
    x"33333333"
);

-- =====

-- TEST 6: AHB Write followed immediately by AHB Read
-- Write 0xABCD1234 then read it back
-- =====
```

```
report "=== TEST 6: AHB Write then Read same address ===" severity note;
do_ahb_write(x"400000C0", x"ABCD1234");
do_ahb_read (x"400000C0");

-- =====
-- END
-- =====

wait for 10 * CLK_PERIOD;
report "=== ALL TESTS COMPLETE ===" severity note;
report "Simulation finished" severity failure;

end process stimulus;

end architecture sim;
```

What Changed vs the Old Testbench

What	Old TB	Fixed TB
AHB Master	Drove signals after fixed wait for delays	Proper model: waits for hgrant=1 AND hready=1 before each phase
APB Slave	Combinational echo of pwrdata	Clocked: responds in SETUP phase, drives prdata in ENABLE phase
Code structure	Everything in one big stimulus process	Separated into ahb_master_model process + stimulus process with helper procedures
Handshaking	None	ahb_start_write/read triggers model, ahb_done tells stimulus when finished
Reset	3 cycles	5 cycles hold + 5 cycles settle after release
Readback reporting	None	report prints hrdata and proc_rdata values to console

